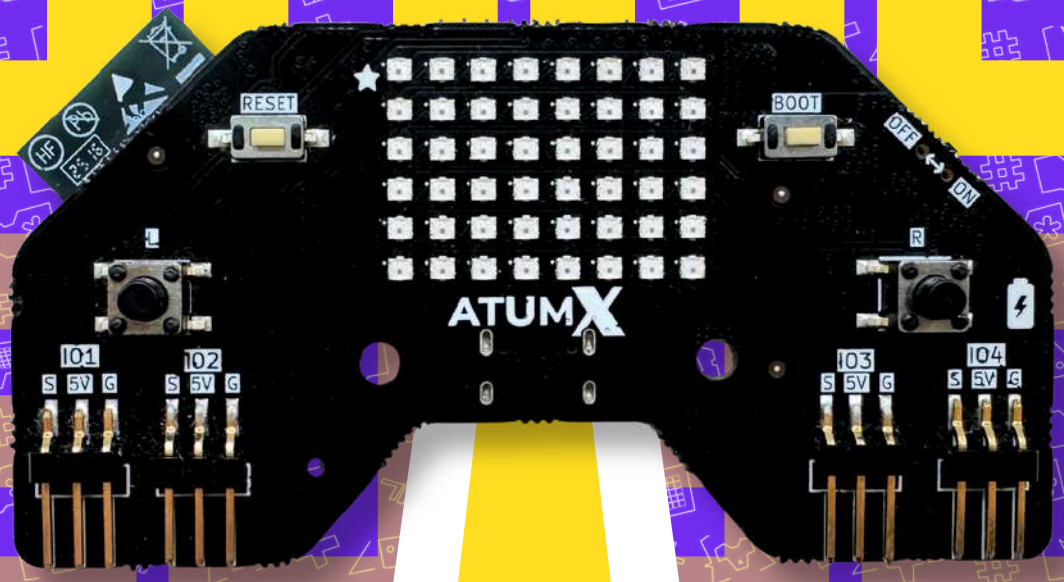
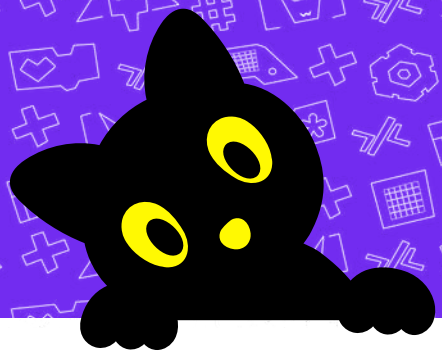
SIRO



User manual and Secrets

Index

App features	4
Connect Subo with App	7
How to run the code?	9
Board Intro	12
Libraries	19
Ultrasonic Sensor	22
IR Sensor	24
LDR Sensor	26
Gas Sensor	28
Touch Sensor	30
Vibration Sensor	32
Sound Sensor	34
DHT11	36

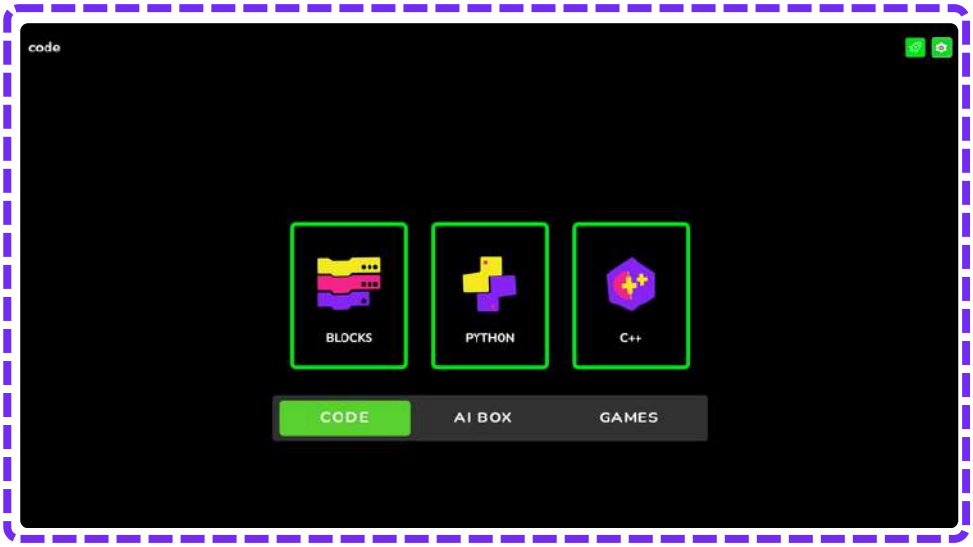


Index

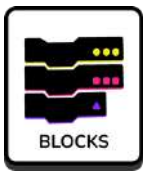
Relay	39
Soil Moisture	41
LCD Display	43
Color Sensor	46
OLED Display	49
Servo Motor 180	51
Servo Motor 360	54
Motor Magic	57
Board Glossary	60

App features

Home Screen



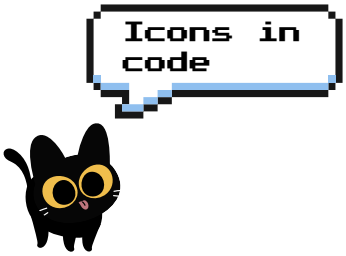
Blocks Mode



A visual drag-and-drop coding system using blocks.

Projects

Opens all the programs and activities you have created or saved.



Python

Text-based coding using the Python programming language.

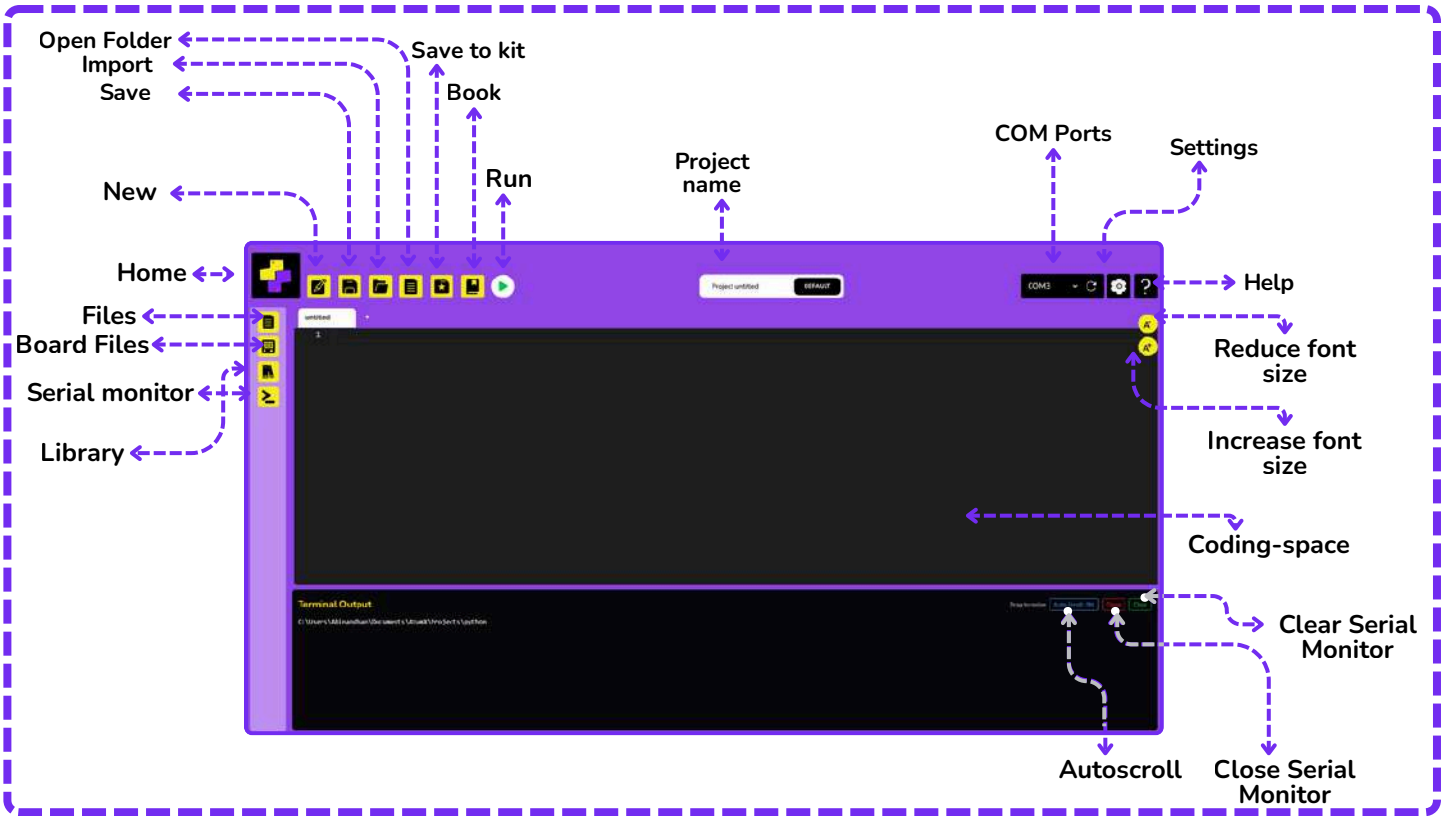
Settings

Used to control Sound, Theme and board options

C++

Coding in the C++ language for faster, performance-based programs.

Parts



Functions

Import

Loads a saved project.

Book

Opens learning resources or documentation.

Save to Kit

Uploads and stores the program directly to our hardware kit

Save

Saves the current project

New

Creates a new project

Home

Returns to the main screen.

Settings

For app related settings.

Run

Executes the program.

Serial monitor

Displays data from the connected device.

Increase font size

Adjusts code view size.

Decrease font size

Adjusts code view size.

Library

Display library files for various sensors that can be used whenever required

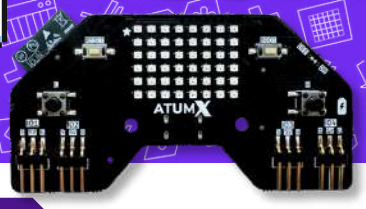
Help

Place to find instructions and help

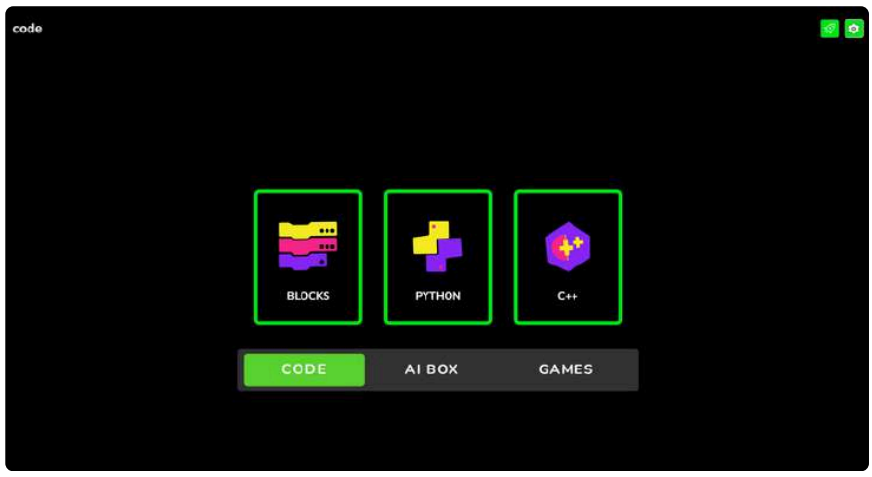
Stop

Stops the execution of code

Connect with the app



Wired mode :

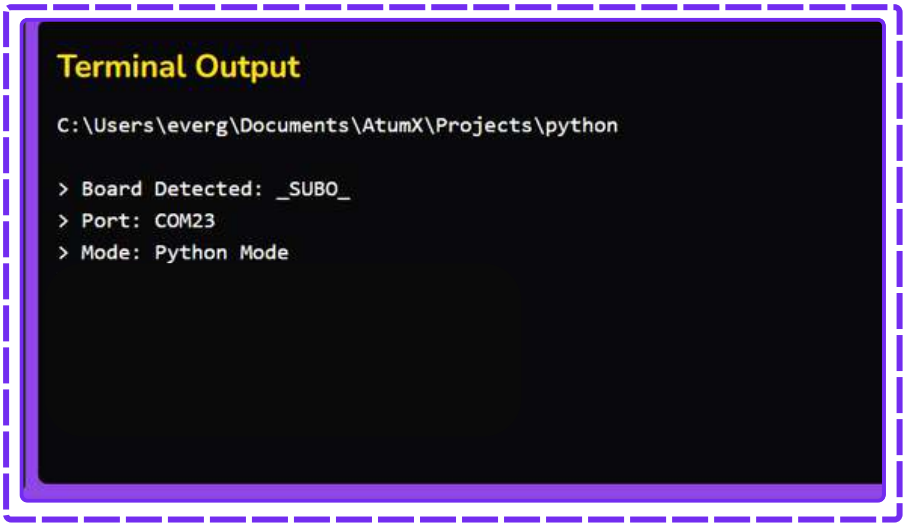


Step 1:

Open the **code app** and select **Python** in the Code.

Step 2:

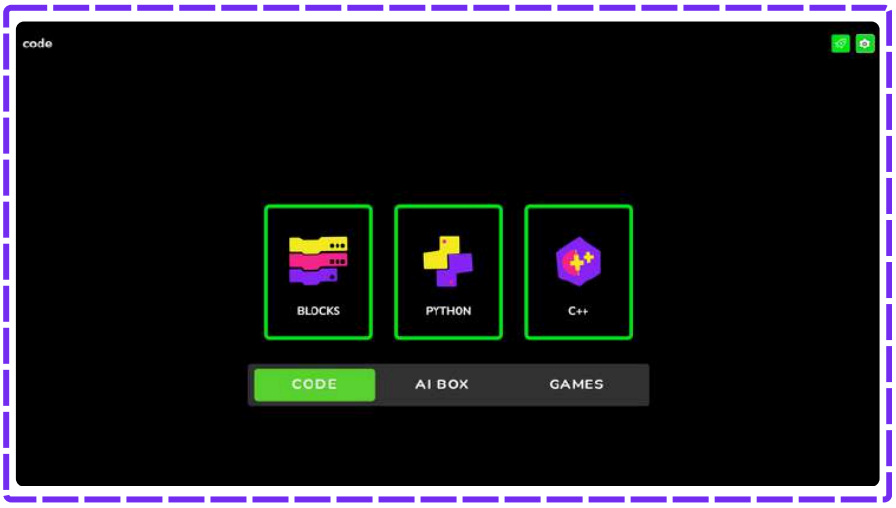
Click on the **refresh** button in the COM ports section on the top right corner.



The COM Ports appear and the board gets **connected automatically** and the **status** is displayed on the **serial monitor**

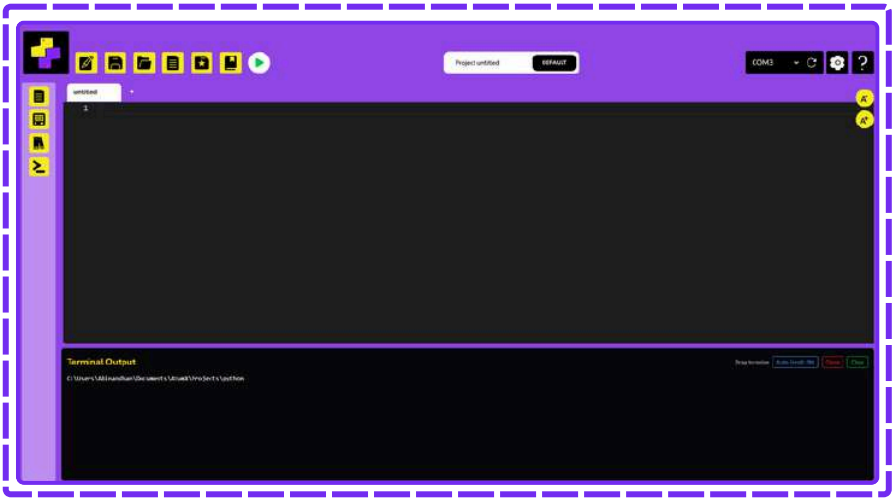
Entering Python

First things first! Make sure to keep the board connected to the PC via USB



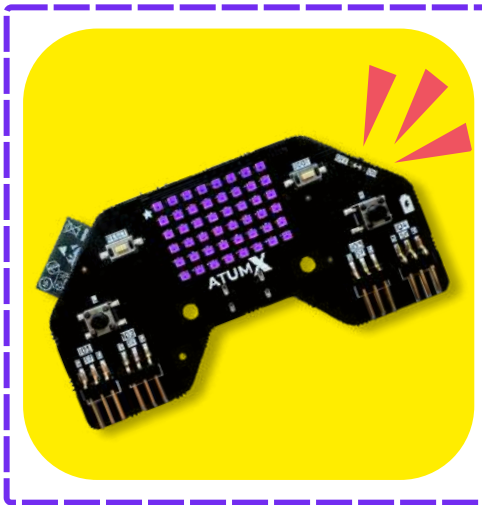
Step 1:

Open the code app and click on the Python code.



Step 2:

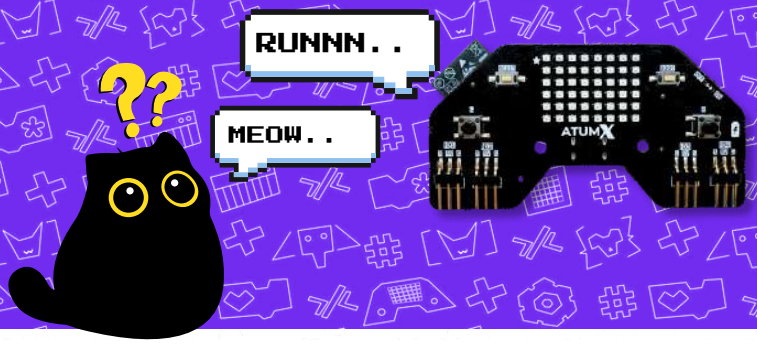
The app is now opened in Python and is ready for coding.



Board LEDs glow in purple indicating a successful switch from blockly to python mode!

The board normally runs on Blockly. Switch it to Python, and the purple lights will light up to let you know it's in Python mode

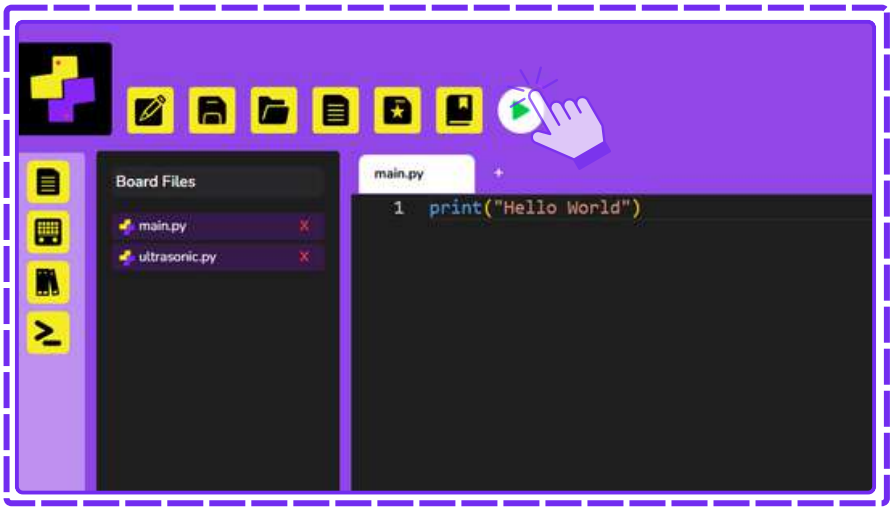
How to run the code?



Before we start getting things done with SUBO, let's learn the two simple ways to program with Python and make things happen. Lets take a look below

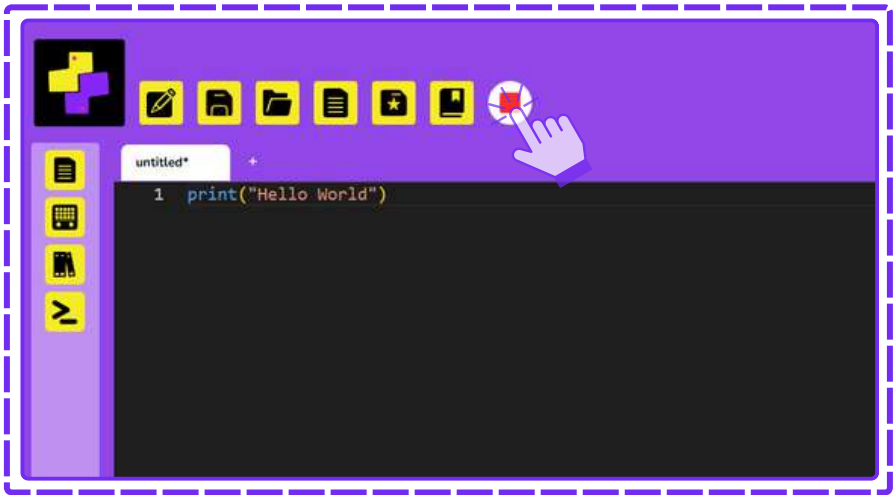
Using Run button

Make sure that the board is connected to the Code app



Step 1:

Click on the green color **run button**.



Step 2:

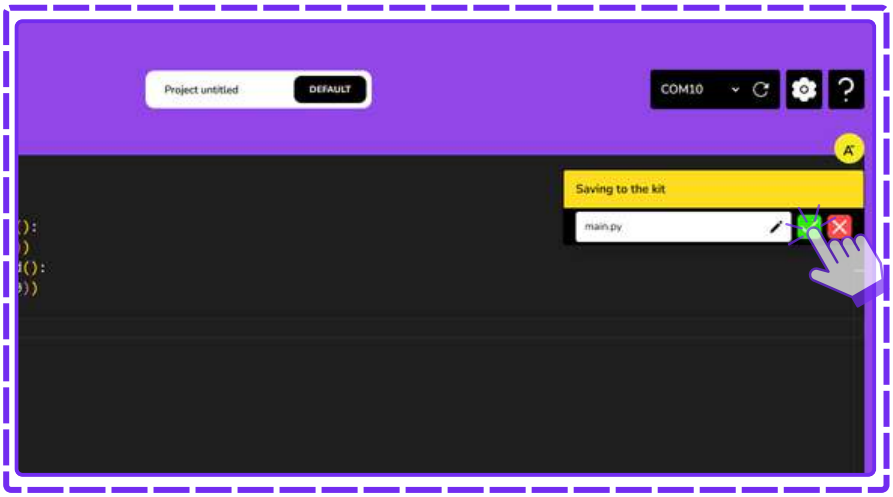
Click on the red color square button to **stop** the code execution

Using Save to Kit

Make sure that the board is connected to code app

Step 1:

Click on the **Save to Kit** icon on the top bar of the code app.



Step 2:

Name your file as **main.py** and **click** the green color **tick icon.**

Step 3:

The file appears on the **board files** section present on the left side of the **Code app** indicating a successful **save**.

Remember

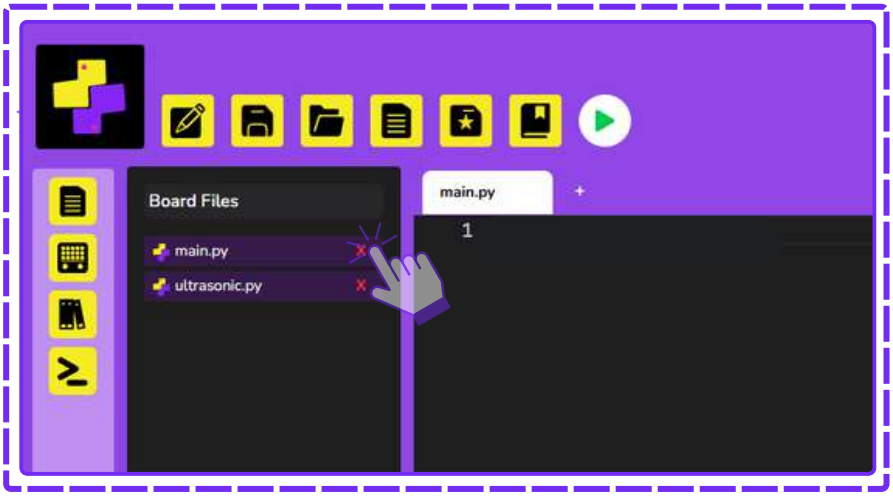
After the program is saved as **main.py**, it runs automatically every time the board is turned on. It can also be run when the SUBO reset button is clicked.

To remove a file saved to kit

Make sure that the board is connected to code app

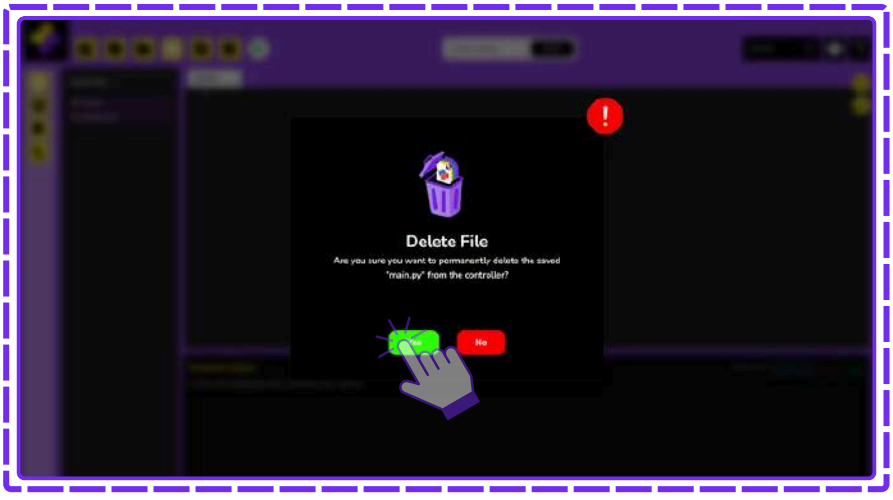
Step 1:

Click on the **Board files** on the left side of the **Code App**



Step 2:

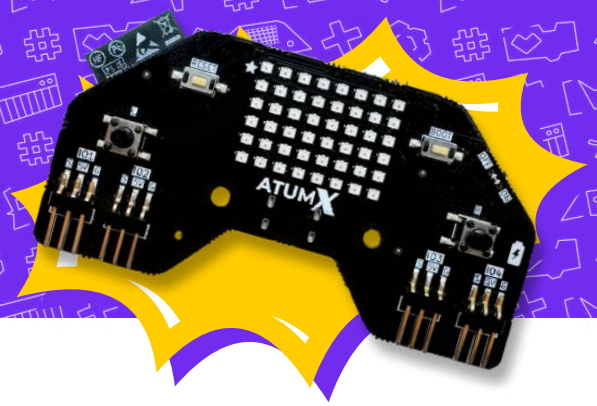
Click on the **X** mark present near the respective file that needs to be deleted.



Step 3:

Click **Yes** on the confirmation message and the file gets deleted successfully.

Board Intro



FIRST THINGS FIRST.
IMPORT SUBO!

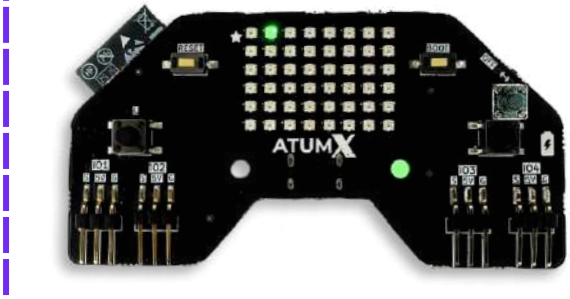
Control the LEDs

Turn on Single LED:

CODE

```
import Subo
Subo.setsingleLED(2, (0, 255, 0))
```

Turns on LED 2 in green color.



Code Explanation

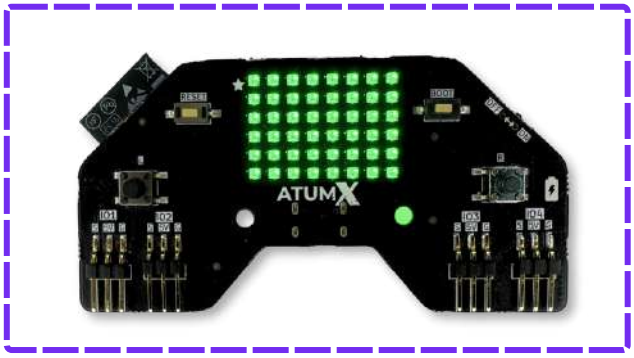
`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.setsingleLED(2, (0, 255, 0))`	Turns ON **LED 2** in **Green** color **(RGB value 255,0,0)**.

Turn on ALL LEDs:

CODE

```
import Subo
Subo.setsingleLED(2, (0, 255, 0))
```

Turns on all LEDs in green color.



Code Explanation

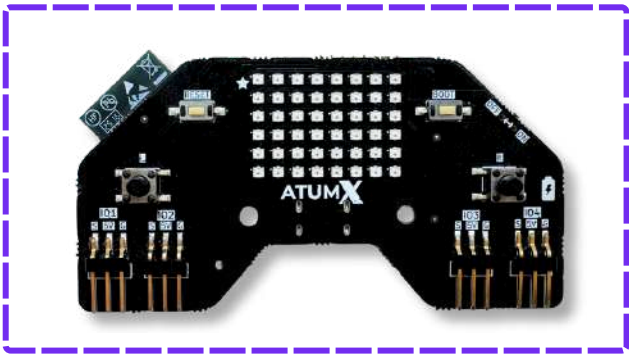
`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.setsingleLED(2, (0, 255, 0))`	Turns ON all **LEDs** in **Green** color **(RGB value 255,0,0)**.

Turn off all LEDs:

CODE

```
import Subo
Subo.stripclear()
```

Turns **OFF** all LEDs.



Code Explanation

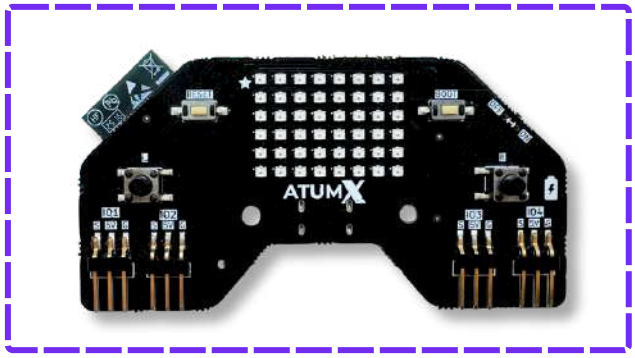
`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.stripclear()`	Turn off all LEDs on the board.

Patterns on LED's

CODE

```
import Subo
Subo.playLEDSeq(1)
```

Turns **OFF** all LEDs.



Code Explanation

`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.playLEDSeq(1)`	Plays different patterns on the LED Strip

```
import Subo
Subo.playLEDSeq(3)
```

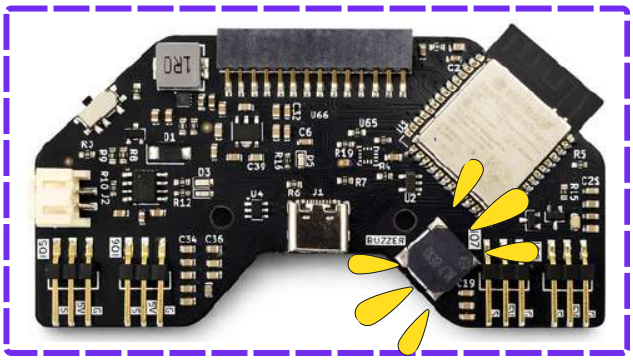
Change the number in **playLEDSeq()** and see how the sequence changes. What happens with 1? What about 5?

try me!

Lets try this code and have some extra fun

Play buzzer:

Buzzer buzzes with the given frequency



CODE

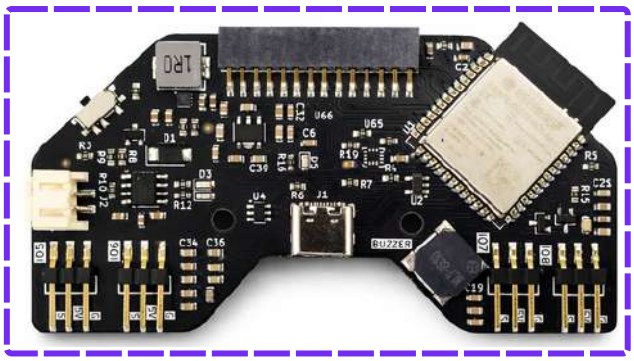
```
import Subo
Subo.playTone(500, 2.0)
```

Code Explanation

`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.playTone(500, 2.0)`	This line makes the buzzer buzz at **500 Hz** for **2.0 seconds**.

Stop a buzzer:

Sends signal to the buzzer to stop buzzing



CODE

```
import Subo
Subo.stopBuzzer()
```

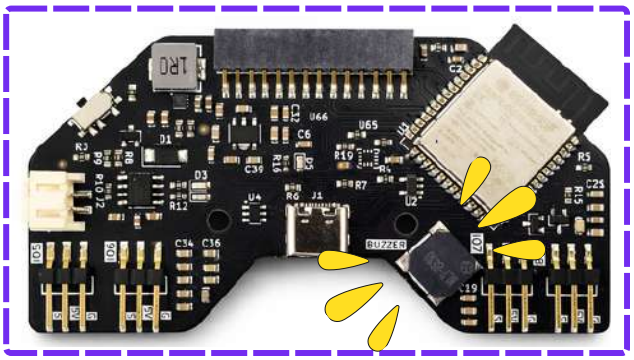
Code Explanation

`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.stopBuzzer()`	**Stops the buzzer** from buzzing

Play buzzer sequence:

CODE

```
import Subo
Subo.playBuzSeq(3)
```



Buzzer buzzes with the given sequence

Code Explanation

`import Subo`	**Import Subo** loads the custom Subo board library to control LEDs, buzzers and IOs.
`Subo.playBuzSeq(3)`	This line makes the buzzer play a built-in sequence of tones, where **3** is the sequence number.

```
import Subo
Subo.playBuzSeq(3)
```

Change the number in **playBuzSeq()** and see how the sequence changes. What happens with 1? What about 5?

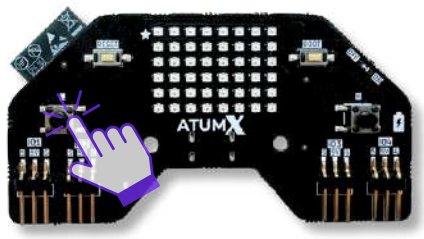
try me!

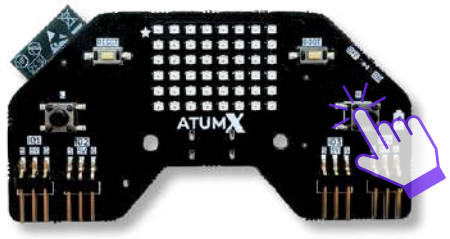
Lets try this code and have some extra fun

Control Buttons

CODE

```
import Subo
import time
while True:
    if Subo.isLeftButtonClicked():
        print("Left Button is Pressed")
    if Subo.isRightButtonClicked():
        print("Right Button is Pressed")
    time.sleep(0.1)
```





The output for the above program can be checked on the serial monitor

Code Explanation

`import Subo` `import time`	Importing the necessary libraries needed to control button in our Subo board.

`while True:`
`  if Subo.isLeftButtonClicked():`
`  print("Left Button is Pressed")`
`  if Subo.isRightButtonClicked():`
`  print("Right Button is Pressed")`
	This loop constantly checks if the user is pressing either the left or right button in Subo board.
	
	If the user presses a button, it prints **"Left button pressed"** or **"Right button pressed"** based on the button pressed by the user on the serial monitor.

`time.sleep(0.1)`	**Pauses** the execution of the current thread for **0.1 seconds**

Toggle colors based on button press:

CODE

```
import Subo
import time
while True:
    if Subo.isLeftButtonClicked():
        Subo.setA11LED((255, 0,0))
    if Subo.isRightButtonClicked():
        Subo.setAl1LED((0, 255, 0))
time.sleep(0.1)
```

All **LEDs** glow **RED** in color, when **Button L** is pressed

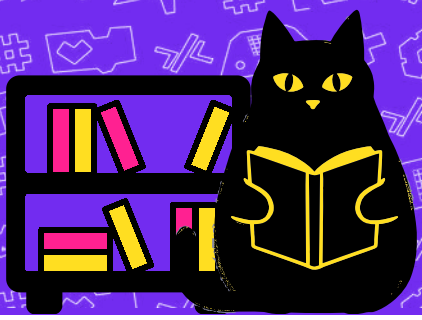
All **LEDs** glow **GREEN** in color, when **Button R** is pressed

`import Subo` `import time`	**Importing** the necessary libraries needed to control button & LEDs in our Subo board.

`while True:`
`  if Subo.isLeftButtonClicked():`
`  Subo.setA11LED((255, 0,0))`
`  if Subo.isRightButtonClicked():`
`  Subo.setAl1LED((0, 255, 0))`
	This loop constantly checks if the user is pressing either the left or right button in Subo board.
	
	If the user presses a button, it makes all LEDs glow **RED** if **Left button** is pressed or all LEDs glow **GREEN** if **Right button** is pressed.

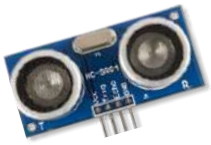
`time.sleep(0.1)`	**Pauses** the execution of the current thread for **0.1 seconds**

Libraries



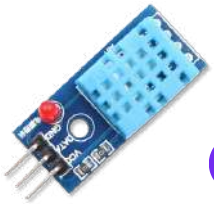
Library

A collection of ready made code that helps hardware modules work with the board.



hcsr04.py

tcs34725.py



dht11.py

motor.py

Pre-Written code that connects sensors and modules to the board

Without the Correct Library, the board cannot understand how to communicate with the sensor.

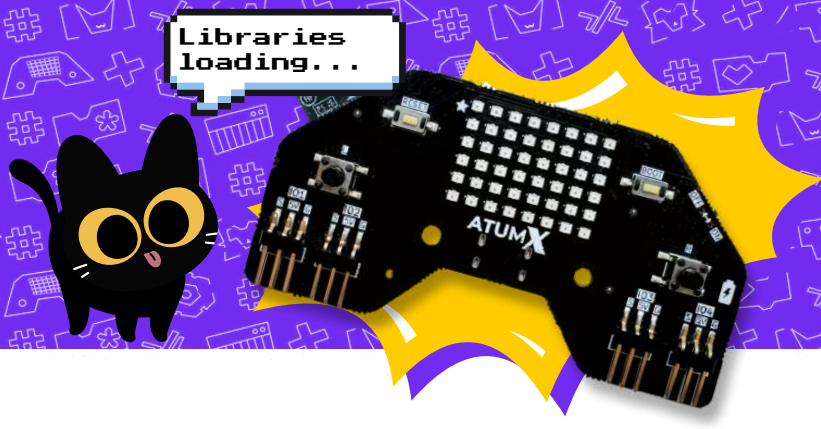
servo.py

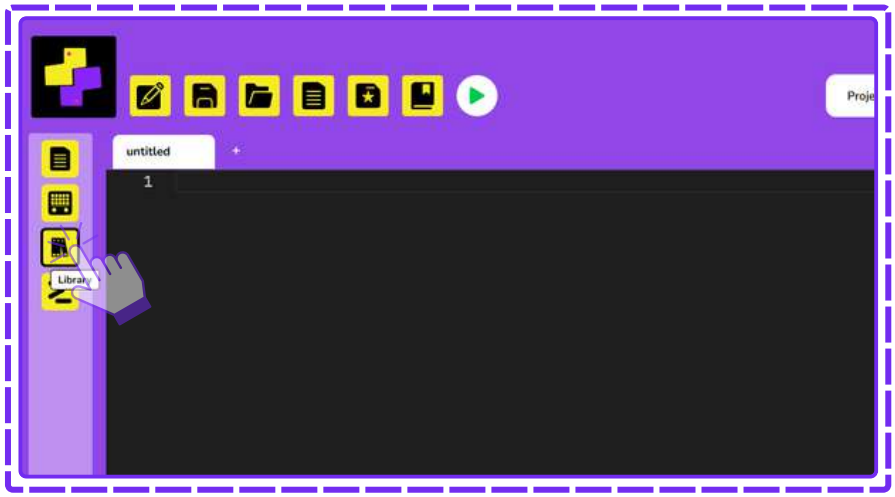
lcd.py/oled.py

Remember!

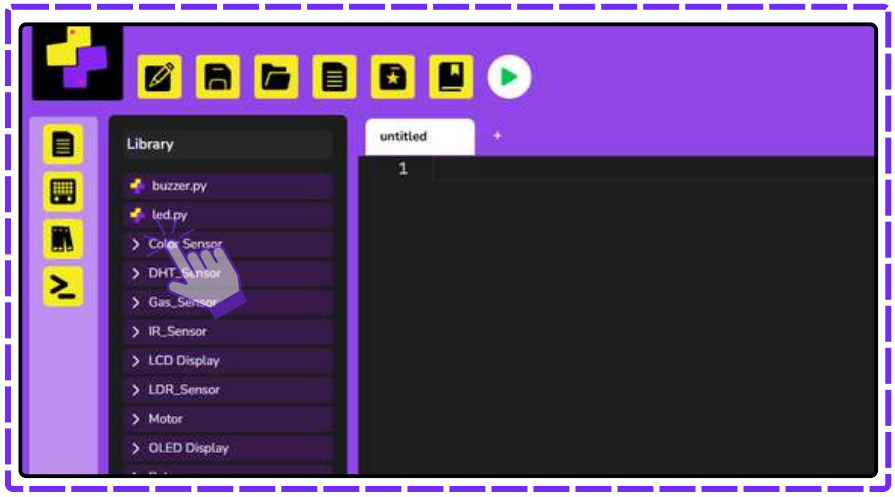
To use each and every sensor, the respective library has to be saved to kit and imported into the main code.

Adding Libraries

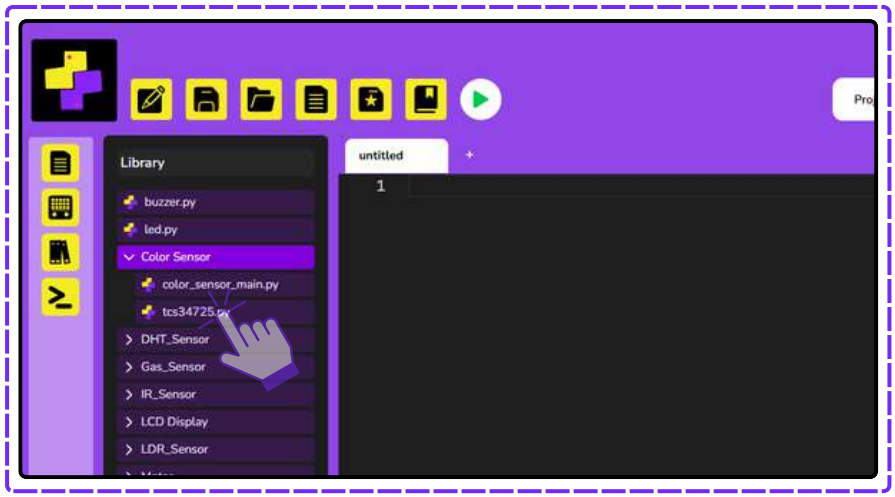




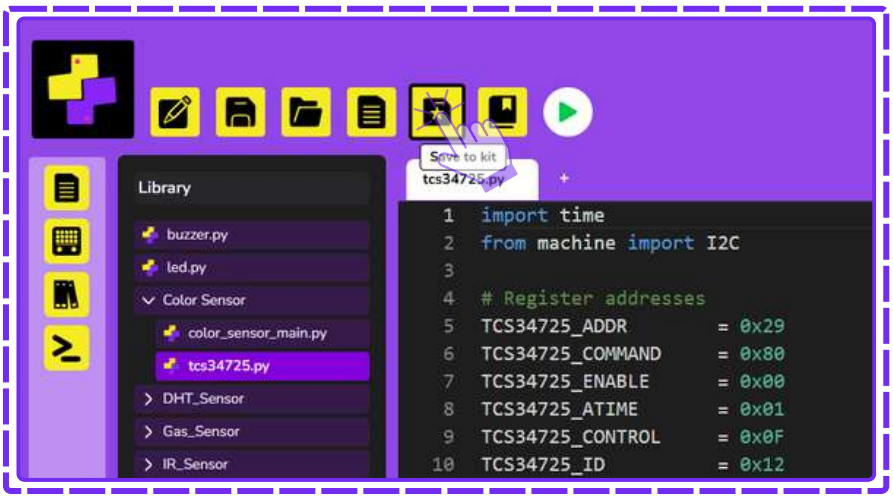
Step 1:
Click on the **Libraries** on the left side of the **Code App**



Step 2:
Click on the **Color Sensor or** any desired sensor.

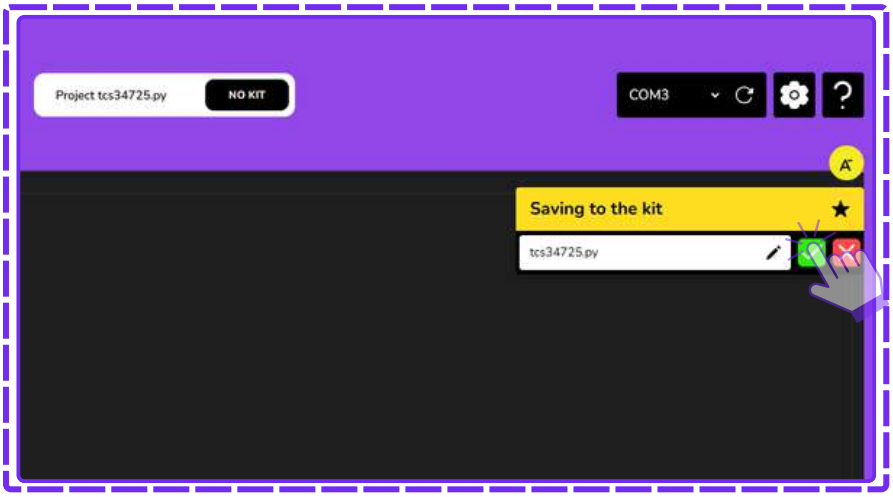


Step 3:
Open the **tcs34725.py** under the **Color Sensor Folder**



Step 4:

After the file is opened, **click** on the **save to kit option.**



Step 5:

Click on the **green check mark** to **save to kit.**

Saving the file as *main.py* runs automatically every time the board is turned on. It can also run when the SUBO reset button is clicked

The library file is saved to the kit. Running the program now with the main code of the sensor will execute the program without any errors.

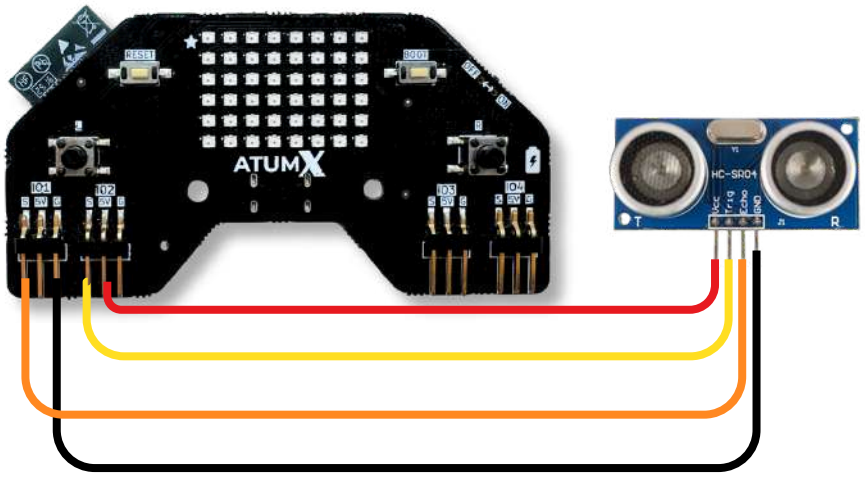
Ultrasonic sensor

Models Compatible

HC-SR04

Pin Connections

Board	Sensor
5V	VCC
GND	Gnd
Connect to **S(IO1)** or any other **available IO pin** defined in the snippet block.	ECHO
Connect to **S(IO2)** or any other **available IO pin** defined in the snippet block.	TRIG



PIN CONNECTIONS

- Power line
- Trigger
- Echo
- Ground line

Python Code

Save ultrasonic.py library file for the Ultrasonic Sensor to the kit before importing in the Python code to avoid errors.

```
import Subo
import time
from ultrasonic import UltraSonic

sensor = UltraSonic(Subo.IO1, Subo.IO2)

while True:
    dist = sensor.distance()
    print("Distance: {:.2f} cm".format(dist))

    time.sleep(0.2)
```

Code explanation

`import Subo`
`import time`
`from ultrasonic import UltraSonic`
	Loads all the necessary libraries needed for the code to work

`sensor = UltraSonic(Subo.IO1, Subo.IO2)`	Sets up the **ultrasonic sensor on IO1** and **IO2** and saves it in the variable **sensor**

`while True:`
`dist = sensor.distance()`
`print("Distance: {:.2f} cm".format(dist))`
	Keeps checking the distance forever using the **while loop** and **prints** it on your screen **every time**

`time.sleep(0.2)`	**Waits** 0.2 seconds before the next reading

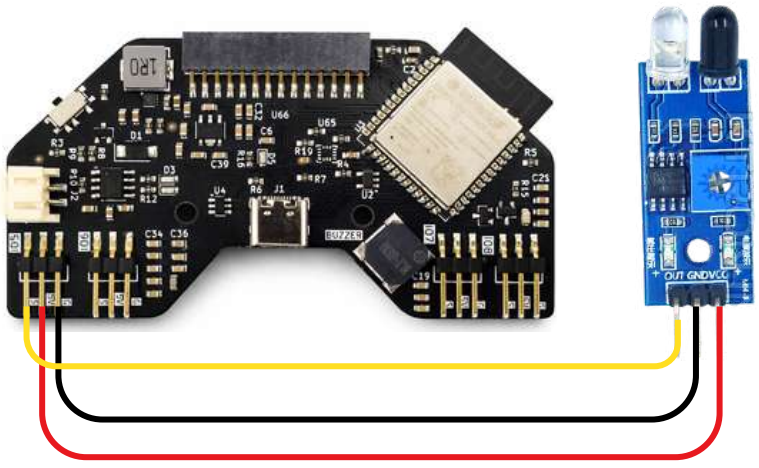
IR sensor

Models Compatible

3 pins- LM393
4 pins- TCRT5000

Pin Connection

Board	Sensor
5 V	VCC
G	GND
Select S(**IO5)** or any other available IO pin defined in the snippet block.	AO/DO



PIN CONNECTIONS

Power line
DO
Ground line

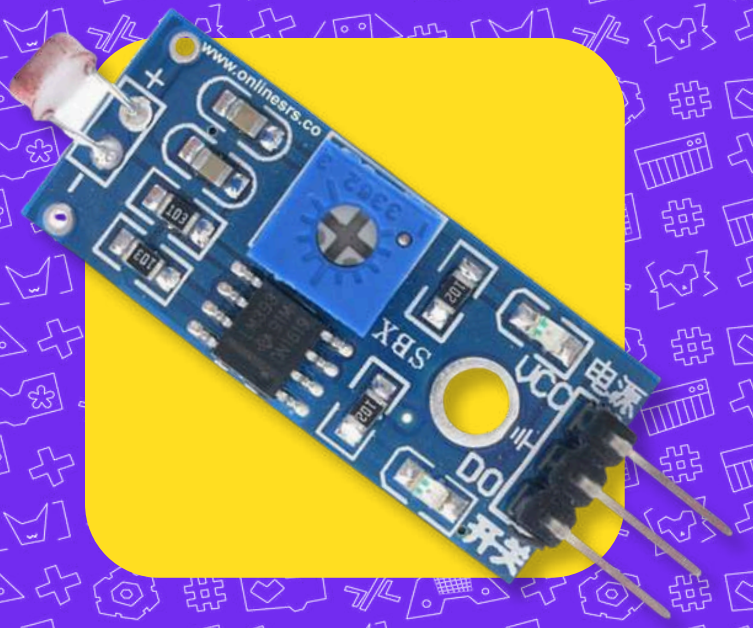
Python Code

```
from machine import Pin
import Subo, time

IR = Subo.IO5
ir = Pin(IR, Pin.IN)

while True:
  print("IR:", ir.value())
  time.sleep(0.5)
```

Code explanation

`from machine import Pin` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work
`IR = Subo.IO5` `ir = Pin(IR, Pin.IN)`	**Setups** the **IR Sensor** on **IO1 pin** of **Subo board**
`while True:` `  print("IR:", ir.value())`	This code keeps printing the values detected by the IR Sensor
`time.sleep(0.5)`	**Waits** 0.5 seconds before the next reading

LDR sensor



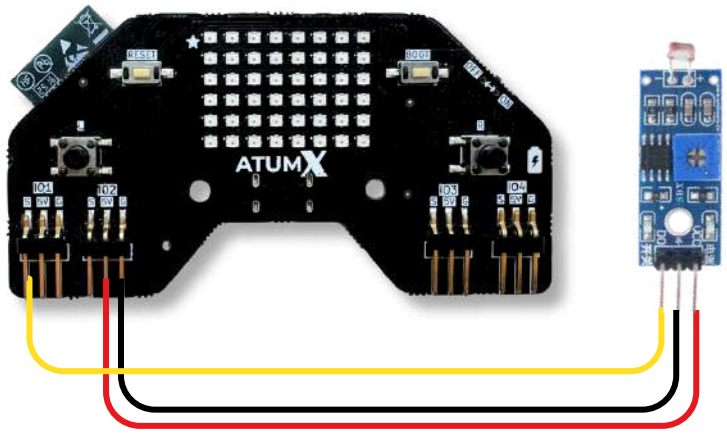
Models Compatible

LM393 based

LDR sensor is an Analog sensor. It works only with IO1, 1O5 and IO8 pins of the SUBO Board.

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO1** or any **available GPIO Pin** defined in the snippet block .	AO/DO



PIN CONNECTIONS

- Power line
- DO
- Ground line

Python Code

```
from machine import Pin, ADC
import Subo, time

LDR = Subo.IO1
ldr = ADC(Pin(LDR))
ldr.atten(ADC.ATTN_11DB)

while True:
    print("LDR:", ldr.read())
    time.sleep(1)
```

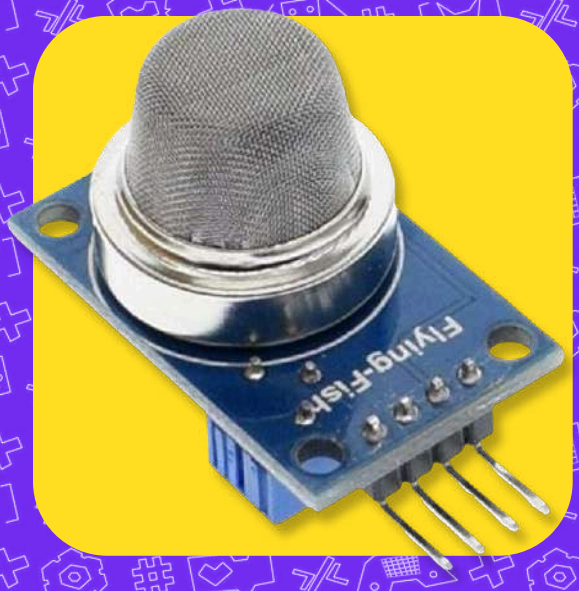
Code explanation

`from machine import Pin, ADC` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work

`LDR = Subo.IO1`
`ldr = ADC(Pin(LDR))`
`ldr.atten(ADC.ATTN_11DB)`
	Sets up the **LDR Sensor** on **IO1 pin** of the board

`while True:` `print("LDR:", ldr.read())`	**Keeps reading** the **light level** forever and **prints it** on your screen everytime using While loop
`time.sleep(1)`	**Waits 1 second** before measuring the next reading

Gas Sensor



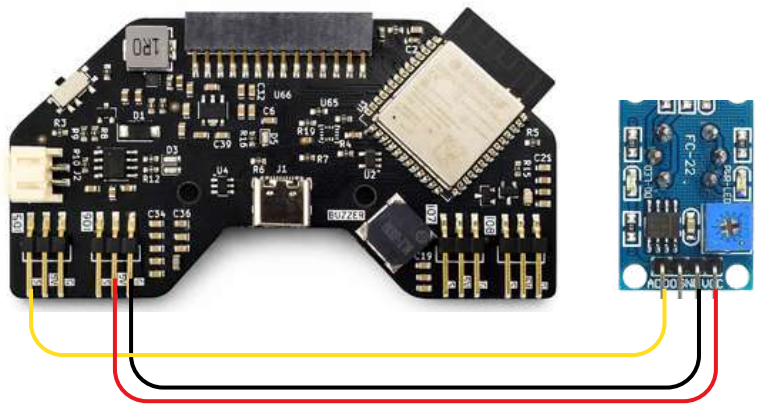
Models Compatible

MQ-2

Pin Connection

> Gas sensor is an Analog sensor. It works only with IO1, 1O5 and IO8 pins of the SUBO Board.

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO5** or any **available GPIO Pin** defined in the snippet block .	AO/DO



PIN CONNECTIONS

- Power line
- DO
- Ground line

Python Code

```
from machine import Pin, ADC
import Subo, time

GAS = Subo.IO5
gas = ADC(Pin(GAS))
gas.atten(ADC.ATTN_11DB)

while True:
    print("Gas:", gas.read())
    time.sleep(1)
```

Code explanation

`from machine import Pin, ADC` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work

`GAS = Subo.IO5`
`gas = ADC(Pin(GAS))`
`gas.atten(ADC.ATTN_11DB)`
	Sets up the **Gas Sensor** on **IO1 pin** of the board

`while True:` `print("Gas:", gas.read())`	**Keeps reading** the gas **level** forever and **prints it** on your screen everytime using **While loop**
`time.sleep(1)`	**Waits 1 second** before the next reading

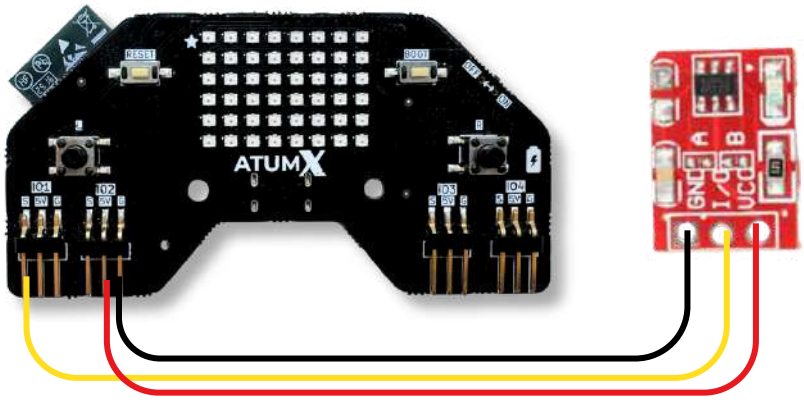
Touch Sensor

Models Compatible

TTP223B : 1 Channel (blue)
TTP223R : 1 Channel (red)

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO1** or any **available GPIO Pin** defined in the snippet block .	AO/DO



PIN CONNECTIONS

Power line
DO
Ground line

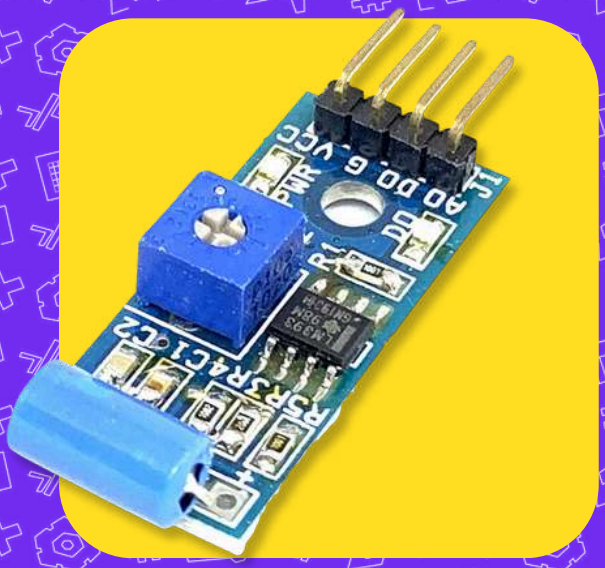
Python Code

```
from machine import Pin
import Subo, time

TOUCH = Subo.IO1
touch = Pin(TOUCH, Pin.IN)

while True:
    print("Touch:", touch.value())
    time.sleep(0.5)
```

Code explanation

`from machine import Pin` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work
`TOUCH = Subo.IO1` `touch = Pin(TOUCH, Pin.IN)`	Setups the touch pin on **IO1** of the SUBO boards
`while True:` `  print("Touch:", touch.value())`	Continuously **prints** the **touch value** that is detected by the **touch sensor** using the **While loop**
`  time.sleep(0.5)`	**Waits** 0.5 seconds before the next reading

Vibration Sensor

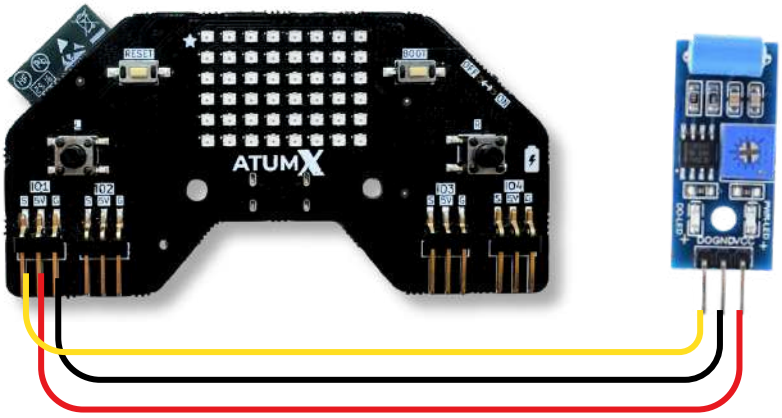


Models Compatible

SW-420

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO1** or any **available GPIO Pin** defined in the snippet block .	AO/DO



PIN CONNECTIONS

Power line
DO
Ground line

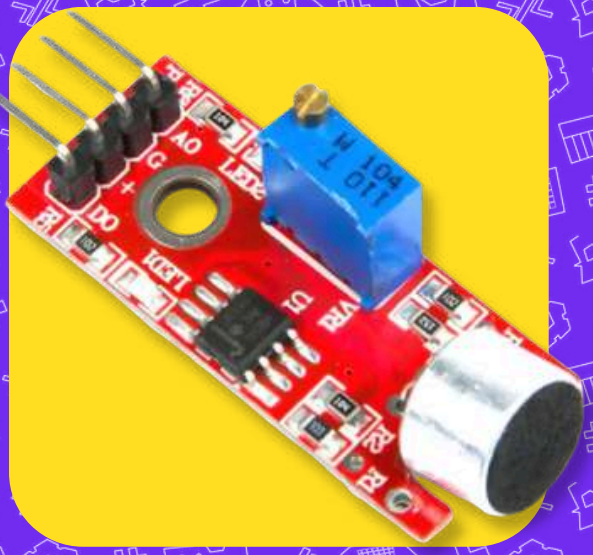
Python Code

```
from machine import Pin
import Subo, time

VIB = Subo.IO1
vib = Pin(VIB, Pin.IN)

while True:
    print("Vibration:", vib.value())
    time.sleep(0.5)
```

Code explanation

`from machine import Pin` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work
`VIB = Subo.IO1` `vib = Pin(VIB, Pin.IN)`	Setups **Vibration Sensor** on **IO1** pin of the board
`while True:` `print("Vibration:", vib.value())`	Keeps checking the **vibration sensor** forever using **While loop** and **prints** the **Vibration value**
`time.sleep(0.5)`	**Waits** 0.1 seconds before the next reading

Sound Sensor

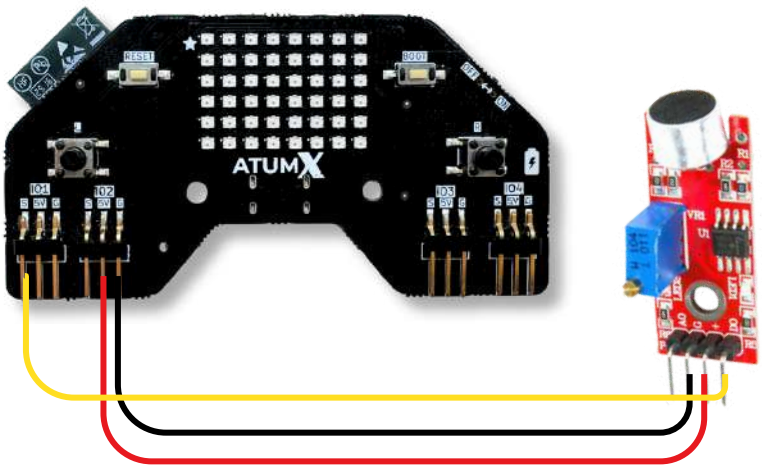


Models Compatible

LM393 op-amp

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO1** or any **available GPIO Pin** defined in the snippet block .	AO/DO



PIN CONNECTIONS

Power line

DO

Ground line

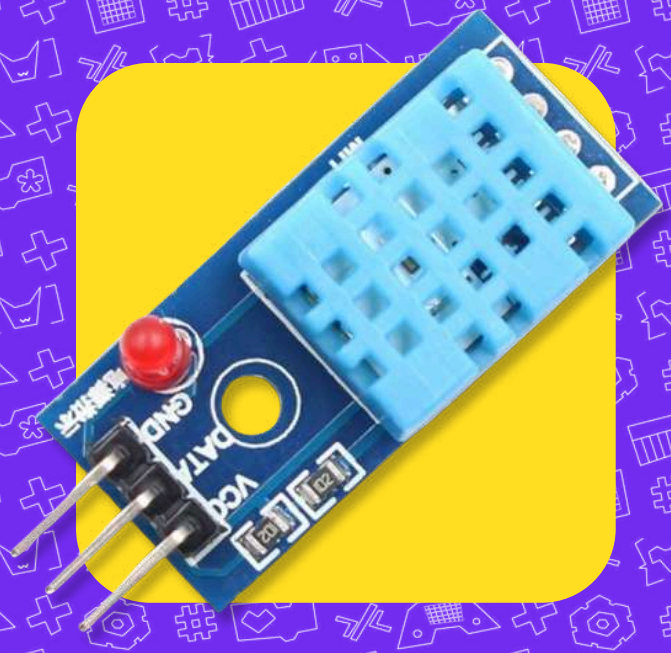
Python Code

```
from machine import Pin
import Subo, time

SOUND = Subo.IO1
sound = Pin(SOUND, Pin.IN)

while True:
    print("Sound:", sound.value())
    time.sleep(0.5)
```

Code explanation

`from machine import Pin` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work
`SOUND = Subo.IO1` `sound = Pin(SOUND, Pin.IN)`	Setups the Sound Sensor on **IO1** pin of the board
`while True:` `  print("Sound:", sound.value())`	**Keeps** checking the **sound sensor** forever using **While loop** and **prints** the **Sound values** from the sensor
`time.sleep(0.5)`	**Waits** 0.5 seconds before the next reading

DHT11

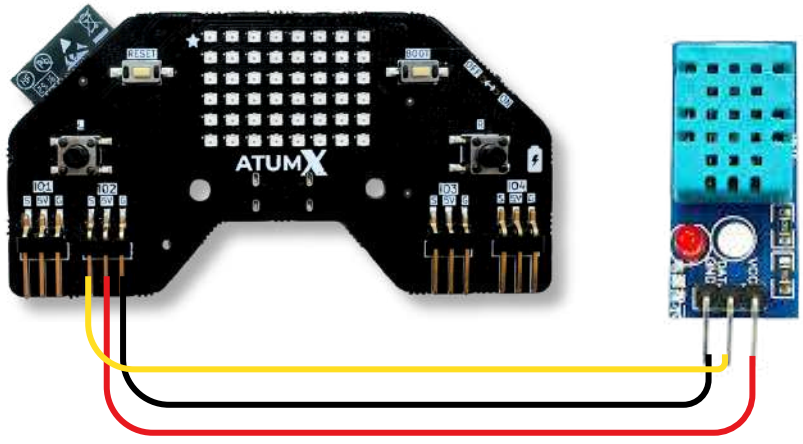


Models Compatible

DHT11

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO2** or any **available GPIO Pin** defined in the snippet block .	DATA



PIN CONNECTIONS

- Power line
- DO
- Ground line

Python Code

Save dht11.py library file for the DHT Sensor to the kit before importing in the Python code to avoid errors.

```
from machine import Pin
from time import sleep
import dht
import Subo

sensor = dht.DHT11(Pin(Subo.IO2))
print("DHT Sensor Test Started...")

while True:
  try:
    sleep(2) # DHT needs delay between readings
    sensor.measure()

    temp = sensor.temperature()
    hum = sensor.humidity()
    temp_f = temp * (9/5) + 32

    print("Temperature: {:.1f} C".format(temp))
    print("Temperature: {:.1f} F".format(temp_f))
    print("Humidity: {:.1f} %".format(hum))
    print("-----------------------------")

  except OSError:
    print("Failed to read sensor. Check wiring!")
```

Code explanation

`from machine import Pin`
`from time import sleep`
`import dht`
`import Subo`
	Loads all the necessary libraries needed for the code to work

`sensor = dht.DHT11(Pin(Subo.IO2))`	**Setups** the **DHT Sensor** on **IO2 pin** of the board

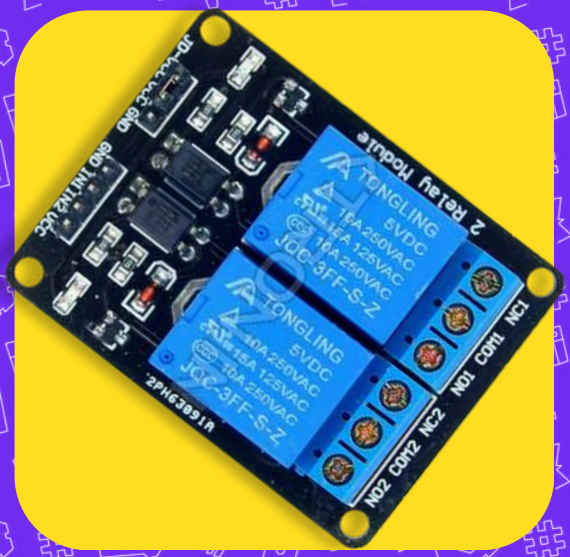
`while True:`
` try:`
`  sleep(2) # DHT needs delay between readings`
`  sensor.measure()`
 	
`  temp = sensor.temperature()`
`  hum = sensor.humidity()`
`  temp_f = temp * (9/5) + 32`
 	
`  print("Temperature: {:.1f} C".format(temp))`
`  print("Temperature: {:.1f} F".format(temp_f))`
`  print("Humidity: {:.1f} %".format(hum))`
`  print("----------------------------")`
	Keeps checking **Temperature value** and **Humidity value** from the **DHT Sensor** and **prints** them continuously using the While loop

Remember!

`temp_f = temp * (9/5) + 32` is the formula used to convert Celsius to Fahrenheit

`{:.1f}` is used to round of the temperature value to 1 decimal point

Relay Module - 2 Relay

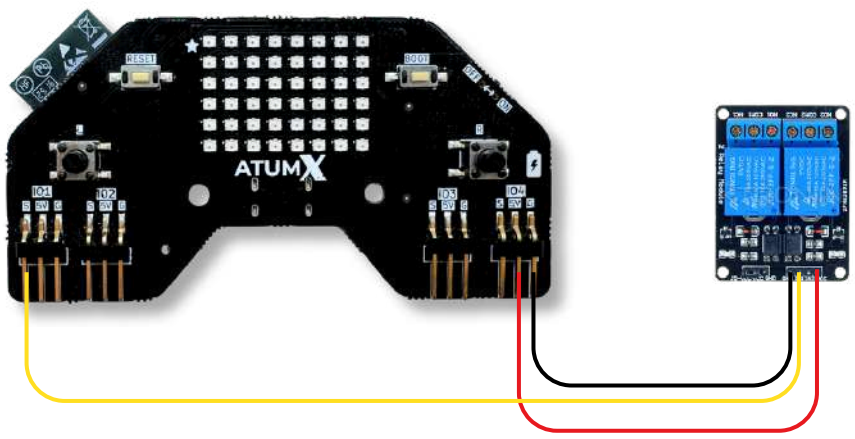


Models Compatible

Optocoupler based relay

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO1** and **IO2** or any **available GPIO Pin** defined in the snippet block .	IN1



PIN CONNECTIONS

- Power line
- Relay
- Ground line

Python Code

```
from machine import Pin
import Subo, time

RELAY_PIN = Subo.IO1
relay = Pin(RELAY_PIN, Pin.OUT)

while True:
  print("Relay ON")
  relay.value(1)
  time.sleep(2)
  print("Relay OFF")
  relay.value(0)
  time.sleep(2)
```

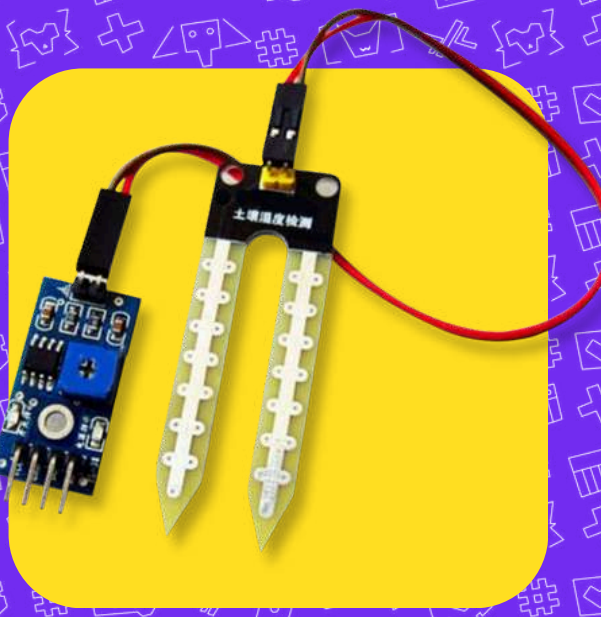
Code explanation

`from machine import Pin` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work
`RELAY_PIN = Subo.IO1` `relay = Pin(RELAY_PIN, Pin.OUT)`	**Keeps** running the relay switching **forever** using a **while** loop

`while True:`
`print("Relay ON")`
`relay.value(1)`
`print("Relay OFF")`
`relay.value(0)`
	Turns both relays **ON** by writing 1 to pins IO1 and IO2 and prints "Relay 1 ON" and "Relay 2 ON"

`time.sleep(0.1)`	Waits 0.1 seconds before the next reading

Soil moisture sensor



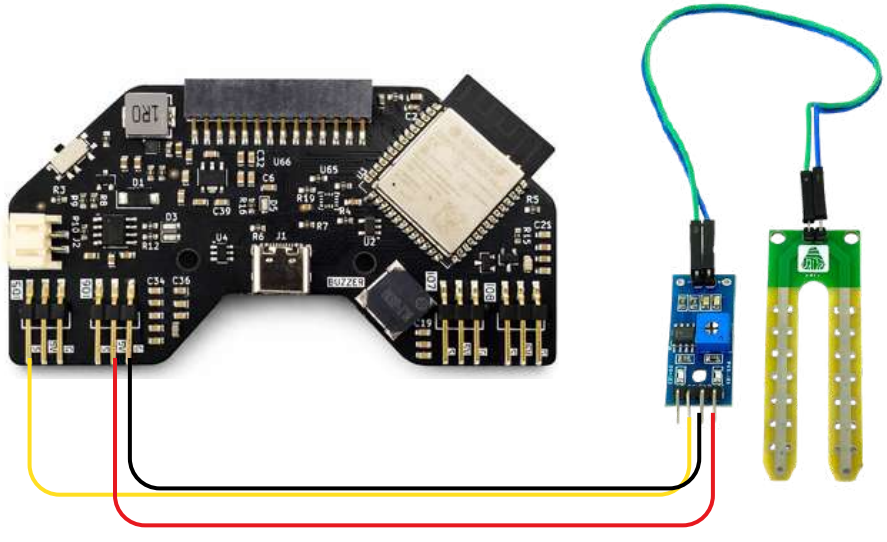
Models Compatible

FC-28 with LM293,
Capacitance based soil sensor

Pin Connection

Board	Sensor
In Board, connect to **any 5V Pins**	VCC
Connect to any **Ground pins**	GND
Connect to **IO5** or any **available GPIO Pin** defined in the snippet block .	AO/DO

Soil moisture sensor is an Analog sensor. It works only with IO1, 1O5 and IO8 pins of the SUBO Board.



PIN CONNECTIONS

- Power line
- DO
- Ground line

Python Code

```
from machine import Pin, ADC
import Subo, time

SOIL = Subo.IO5
soil = ADC(Pin(SOIL))
soil.atten(ADC.ATTN_11DB)

while True:
    print("Soil:", soil.read())
    time.sleep(1)
```

Code explanation

`from machine import Pin, ADC` `import Subo, time`	**Loads** all the necessary libraries needed for the code to work

`SOIL = Subo.IO5`
`soil = ADC(Pin(SOIL))`
`soil.atten(ADC.ATTN_11DB)`
	Setups the **Soil moisture sensor** on the **IO5** pin of the board,

`while True:` `print("Soil:", soil.read())`	Continuously reads and checks for **moisture values** using **While loop** from the soil moisture sensor
`time.sleep(1)`	**Waits 1 second** before the next reading

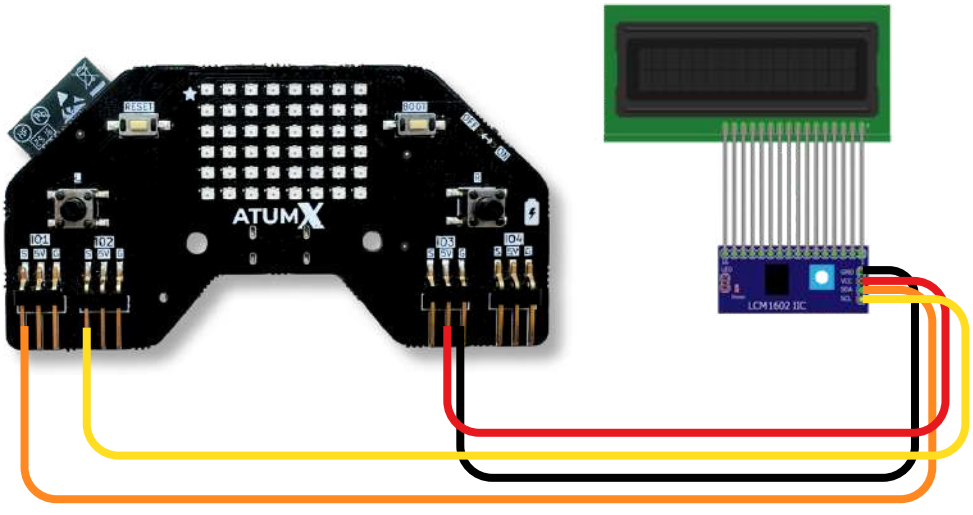
LCD Display

Models Compatible

16x2 LCD1602 Display with I2C

Pin Connection

Board	Sensor
5V	VCC
GND	GND
Connect to **S(IO1)** or any other **available IO pin** defined in the snippet block.	SCL
Connect to **S(IO2)** or any other **available IO pin** defined in the snippet block.	SDA



Python Code

Save lcd.py library file for the LCD Displayto the kit before importing in the Python code to avoid errors.

```
import time
from machine import I2C, Pin
import Subo
from lcd import LCD1602I2C

i2c = I2C(0, sda=Pin(Subo.IO2), scl=Pin(Subo.IO1), freq=400000)

lcd = LCD1602I2C(i2c, i2c_addr=0x27)

lcd.set_cursor(1, 1)
lcd.print("Hello, Subo!")

lcd.set_cursor(1, 2)
lcd.print("I2C LCD Ready")

time.sleep(2)
lcd.clear()

lcd.set_cursor(1, 1)
lcd.print_scroll("Scroll Left", "Left")
```

Code explanation

```
import time
from machine import I2C, Pin
import Subo
from lcd import LCD1602I2C
```

Loads all the necessary libraries needed for the code to work

```
i2c = I2C(0, sda=Pin(Subo.IO2), scl=Pin(Subo.IO1), freq=400000)
lcd = LCD1602I2C(i2c, i2c_addr=0x27)
```

Sets up the **LCD Display** on **IO1** and **IO2** **pin** of the Board

```
lcd.set_cursor(1, 1)
lcd.print("Hello, Subo!")

lcd.set_cursor(1, 2)
lcd.print("I2C LCD Ready")
```

Displays the specified text on the **LCD screen**

```
lcd.set_cursor(1, 1)
lcd.print_scroll("Scroll Left", "Left")
```

Displays the specified text in a **left scrolling** manner

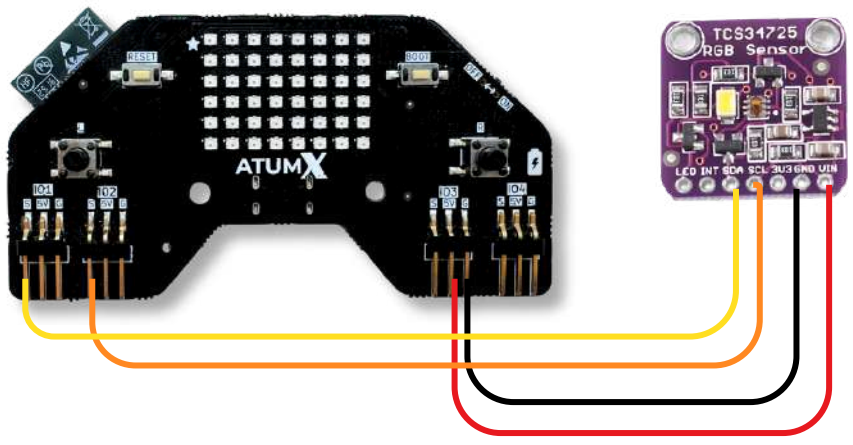
Colour sensor

Models Compatible

TCS34725

Pin Connection

Board	Sensor
5V	VCC
GND	GND
Connect to **S(IO1)** or any other **available IO pin** defined in the snippet block.	SDA
Connect to **S(IO2)** or any other **available IO pin** defined in the snippet block.	SCL



PIN CONNECTIONS

- Power line
- Trigger
- Echo
- Ground line

Python Code

Save tcs34725.py library file for the Color Sensor to the kit before importing in the Python code to avoid errors.

```
import time
from machine import I2C, Pin
import Subo
from tcs34725 import TCS34725

i2c = I2C(0, sda=Pin(Subo.IO1), scl=Pin(Subo.IO2), freq=400000)

sensor = TCS34725(i2c)
print("TCS34725 ready!")

while True:
  name = sensor.color_name()
  print("Color:", name)

  if name == "Green":
    Subo.set_all_leds(0, 255, 0)
  elif name == "Red":
    Subo.set_all_leds(255, 0, 0)
  elif name == "Blue":
    Subo.set_all_leds(0, 0, 255)
  else:
    Subo.set_all_leds(0, 0, 0)

  time.sleep(0.5)
```

Code explanation

```
import time
from machine import I2C, Pin
import Subo
from tcs34725 import TCS34725
```

Loads all the necessary libraries needed for the code to work

```
i2c = I2C(0, sda=Pin(Subo.IO1), scl=Pin(Subo.IO2), freq=400000)
sensor = TCS34725(i2c)
print("TCS34725 ready!")
```

Setups the **color sensor** on **IO1** and **IO2 pins** of the **board**

```
while True:
 name = sensor.color_name()
 print("Color:", name)

 if name == "Green":
  Subo.set_all_leds(0, 255, 0)
 elif name == "Red":
  Subo.set_all_leds(255, 0, 0)
 elif name == "Blue":
  Subo.set_all_leds(0, 0, 255)
 else:
  Subo.set_all_leds(0, 0, 0)
```

Continuously checks for the color from the **Color sensor** and **displays** the color on the serial monitor using **While loop**. It also **set** the **LED's** on the board to the **specific color**.

```
time.sleep(0.5)
```

Waits 0.5 seconds before the next reading

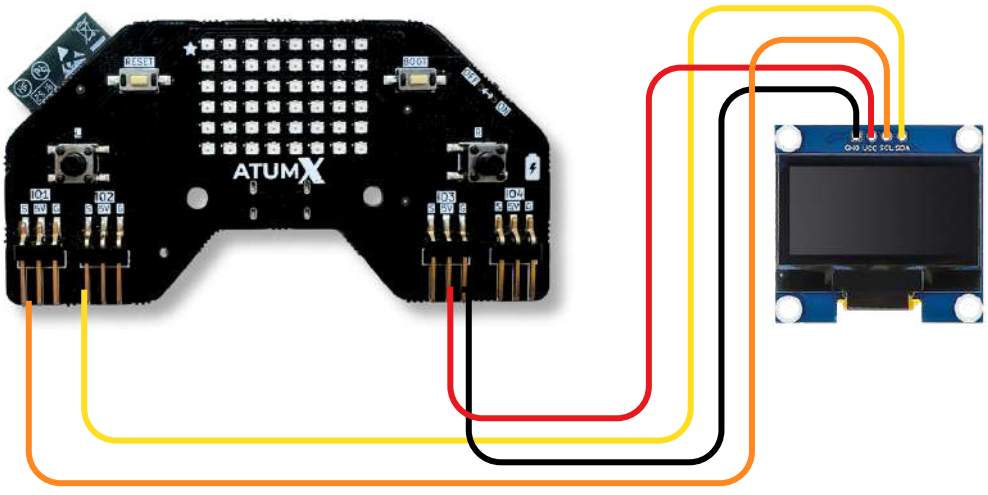
OLED Display

Models Compatible

1.3 inch (I2C) OLED display

Pin Connection

Board	Sensor
5V	VCC
GND	GND
Connect to **S(IO1)** or any other **available IO pin** defined in the snippet block.	SCL
Connect to **S(IO2)** or any other **available IO pin** defined in the snippet block.	SDA



Python Code

```
import Subo
from machine import Pin, I2C
import sh1106

sda_pin = Pin(Subo.IO2)
scl_pin = Pin(Subo.IO1)

i2c = I2C(0, sda=sda_pin, scl=scl_pin)

display = sh1106.SH1106_I2C(128, 64,
i2c)

display.fill(0)
display.text("Subo OLED OK", 0, 0)
display.show()
```

Save sh1106.py library file for the OLED display to the kit before importing in the Python code to avoid errors.

Code explanation

`import Subo`
`from machine import Pin, I2C`
`import sh1106`
	Loads all the necessary libraries needed for the code to work

`sda_pin = Pin(Subo.IO2)`
`scl_pin = Pin(Subo.IO1)`
`i2c = I2C(0, sda=sda_pin, scl=scl_pin)`
	Sets up the I2C communication between the SUBO board and the OLED display using the **SCL** and **SDA** pins

`display = sh1106.SH1106_I2C(128, 64, i2c)`	**Sets up** the **OLED display** with a **size of 128x64 pixels** and saves it as **display** variable

`display.fill(0)`
`display.text("Subo OLED OK", 0, 0)`
`display.show()`
	Displays the desired text on the **OLED display**

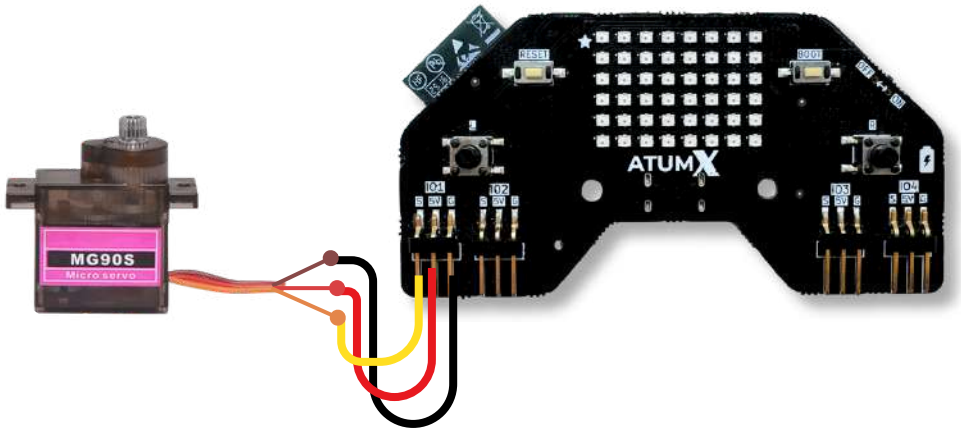
Servo Motor 180°

Models Compatible

Tower Pro MG90 S Micro Gear Servo Motor

Pin Connection

Board	Sensor
Connect to **IO1** or any S(IO) in board defined in the snippet block .	Yellow wire
5V	Red wire
Ground	Brown wire



PIN CONNECTIONS

- Power line
- Ground line
- Signal

Python Code

Save servo.py library file for the Servo Motor to the kit before importing in the Python code to avoid errors.

```
import Subo
import time
from servo import Servo

motor = Servo(Subo.IO1)

while True:
  print("Moving 0 to 180...")
  motor.angle(0)

  time.sleep(1)
  motor.angle(180)
  time.sleep(1)

  print("Spinning...")
  motor.speed(50)
  time.sleep(2)
  motor.speed(0)
```

Code explanation

`import Subo`
`import time`
`from servo import Servo`
	Loads all the necessary libraries needed for the code to work

`motor = Servo(Subo.IO1)`	**Sets up** the servo motor on pin **IO1** with

```
while True:
 print("Moving 0 to 180...")
 motor.angle(0)

 time.sleep(1)
 motor.angle(180)
 time.sleep(1)

 print("Spinning...")
 motor.speed(50)
 time.sleep(2)
 motor.speed(0)
```

The **while loop** keeps the motor moving forever. First it prints "Moving 0 to 180° and swings the motor from 0° to 180° going left to right. It waits 1 second at each position.

Then it **prints** "Spinning..." and spins the motor at 50% speed for 2 seconds before stopping completely with motor.speed(0)

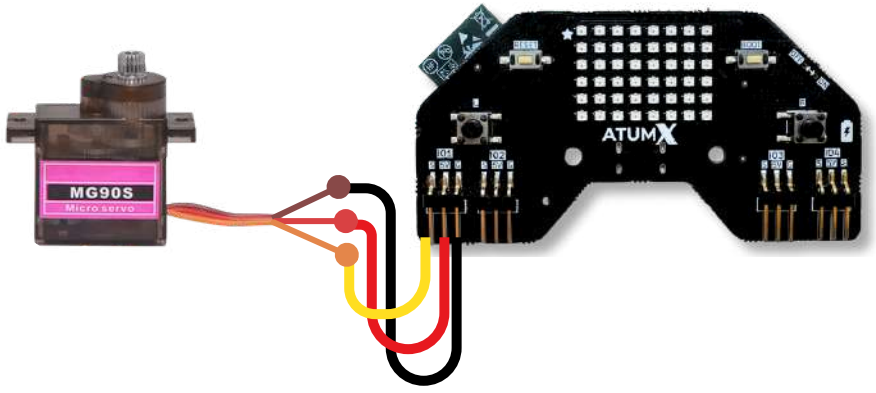
Servo Motor 360°

Models Compatible

Tower Pro MG90 S Micro Gear Servo Motor

Pin Connection

Board	Sensor
Connect to **IO1** or any S(IO) in board defined in the snippet block .	Yellow wire
5V	Red wire
Ground	Brown wire



PIN CONNECTIONS

- Power line
- Ground line
- Signal

Python Code

Save servo360.py library file for the 360 degree servo motor to the kit before importing in the Python code to avoid errors.

```
import Subo
import time
from servo360 import Servo360

motor = Servo360(Subo.IO1)

print("Testing 360 Servo...")

while True:
  print("Forward ->")
  motor.speed(100)
  time.sleep(2)

  print("Stop")
  motor.speed(0)
  time.sleep(2)

  print("<- Backward")
  motor.speed(-50)
  time.sleep(2)
```

Code explanation

`import Subo`
`import time`
`from servo360 import Servo360`
	Loads all the necessary libraries needed for the code to work

`motor = Servo360(Subo.IO1)`	Sets up the 360° servo motor on pin **IO1** and saves it as **motor**

`while True:`
`  print("Forward ->")`
`  motor.speed(100)`
`  time.sleep(2)`
 	
`  print("Stop")`
`  motor.speed(0)`
`  time.sleep(2)`
 	
`  print("<- Backward")`
`  motor.speed(-50)`
`  time.sleep(2)`
	Continuously runs the motor using the While loop.
	
	Firstly it runs the motor at **full speed** in a **forward direction**.
	
	The speed is then changed to **zero** which **stops the motor**.
	
	Later the motor spins in the **reverse direction** at a **speed of 50**.
	
	This stays in a **loop** until the program is stopped manually.

`time.sleep(2)`	**Waits 2 seconds** before **moving the motor**

Motor Magic

Models Compatible

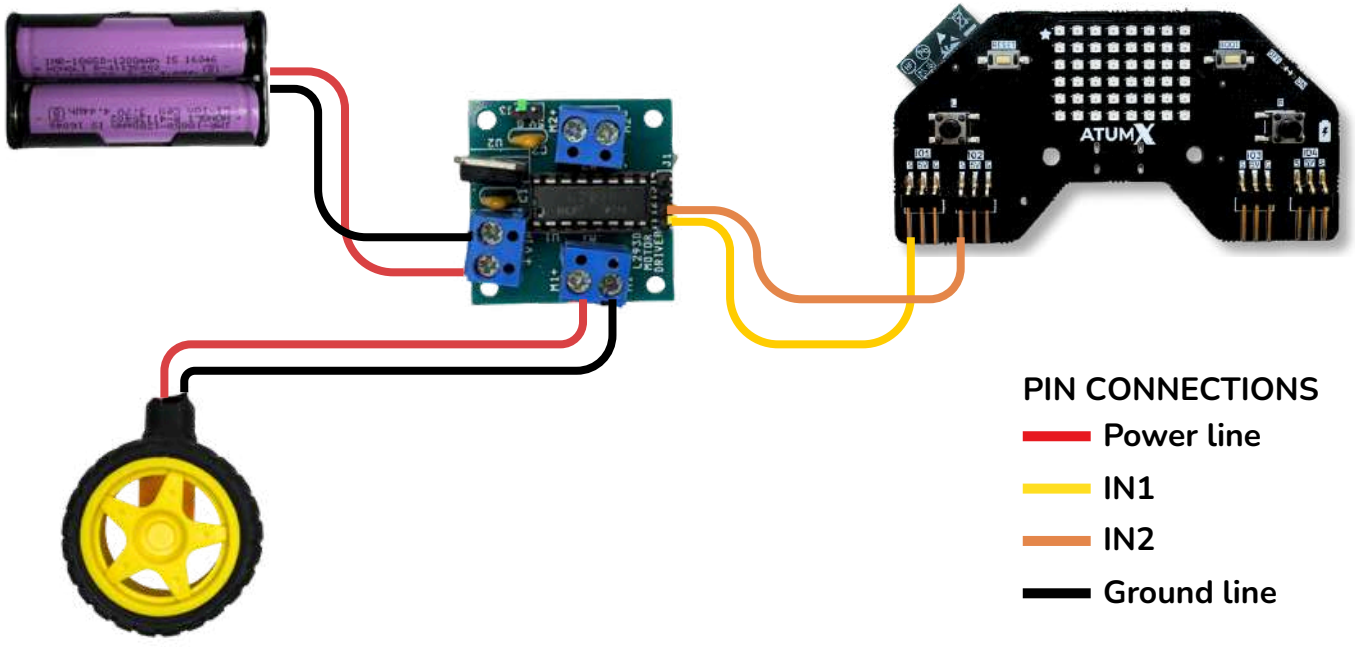
L293D / L298N Motor Driver Module

Pin Connection

Board	Motor Controller
Any GND	GND
IO1 or(any IO pin defined in the snippet block)	IN1
IO2 or (any IO pin defined in the snippet block)	IN2

Motor	Motor Controller
Wire 1	M1 +
Wire 2	M1 -

Double cell battery	Motor Controller
Red Wire	VIN
Black Wire	GND



Python Code

Save servo360.py library file for the 360 degree servo motor to the kit before importing in the Python code to avoid errors.

```
import Subo
from motor import Motor
import time

Motor1 = Motor("Motor1", Subo.IO1, Subo.IO2)

while True:
    Motor1.run("Forward", 80)
    time.sleep(2)

    Motor1.stop()

    Motor1.run("Backward", 50)
    time.sleep(2)

    Motor1.stop()

    Motor1.setPWM(150, 0)
    time.sleep(2)

    Motor1.setPWM(0, 0)
    time.sleep(2)

    Motor1.setPWM(0, 150)
    time.sleep(2)

    Motor1.stop()
```

If you want to connect our board with motor without using USB, Use a JST battery as shown below

Code explanation

`import Subo`
`from motor import Motor`
`import time`
	Loads all the necessary libraries needed for the code to work

`Motor1 = Motor("Motor1", Subo.IO1, Subo.IO2)`	**Keeps** running the relay switching **forever** using a **while** loop

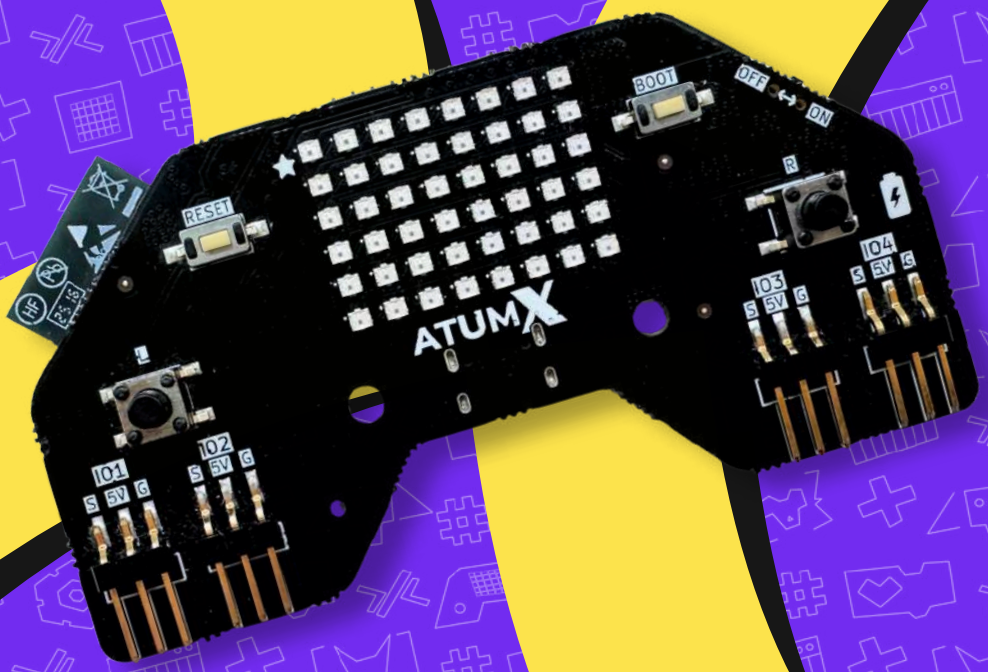
`while True:`
`  Motor1.run("Forward", 80)`
`  time.sleep(2)`
 	
`  Motor1.stop()`
 	
`  Motor1.run("Backward", 50)`
`  time.sleep(2)`
 	
`  Motor1.stop()`
 	
`  Motor1.setPWM(150, 0)`
`  time.sleep(2)`
 	
`  Motor1.setPWM(0, 0)`
`  time.sleep(2)`
 	
`  Motor1.setPWM(0, 150)`
`  time.sleep(2)`
 	
`  Motor1.stop()`
	This runs the motor in a loop
	
	Firstly it **runs** in a **forward direction** at 80% speed.
	
	Then it runs in a **backward direction** at 50% speed.
	
	The **Motor1.setPWM(150, 0)** moves the motor in the **forward direction**.
	
	The **Motor1.setPWN(0, 0)** keeps the **motor stopped**.
	
	The **Motor1.setPWM(0, 150)** runs the motor in a **reverse direction**

Board Glossary

`Subo.setSingleLED(index, color)`	**Set a single LED's color** to an RGB Value provided by the user **Parameters:** • **index -** Zero Index of the LED from the first LED (Marked with a Star) • **color -** Tuple of the RGB Values **Return type:** None
`Subo.setAllLED(color)`	**Set a All LEDs' colors** to an RGB Value provided by the user **Parameters:** **color (tuple) -** Tuple of the RGB Values **Return type:** None
`Subo.playTone(freq, duration)`	**Play a Tone** on the **buzzer** for the **frequency** and **duration** provided by the user **Parameters:** • **freq** (int) - Frequency at which the buzzer should sound • **duration** (float) - duration in seconds **Return type:** None
`Subo.stopBuzzer()`	**Stop a buzzer** that is playing **Return type:** None

Board Glossary

`Subo.stripclear()`	**Turn off all LEDs** on the board **Return type:** None
`Subo.playBuzSeq(id)`	**Play a pre-programmed Buzzer Sequence** **Parameters:** **id** - Play Sequence associated with ID. ID ranges from 1 to 5 **Return type:** None
`Subo.playLEDSeq(id)`	**Play a pre-programmed LED Sequence** **Parameters:** **id** - Play Sequence associated with ID. ID ranges from 1 to 5 **Return type:** None
`Subo.isRightButtonClicked()`	**Get Right Button reading** **Return type:** Reading of Right Button
`Subo.isLeftButtonClicked()`	**Get Left Button reading** **Return type:** Reading of Left Button


RESET
BOOT
OFF
ON
R
ATUMX
IO1
IO2
IO3
IO4
S
5V
G